Finally, we give the following definition which we will use to formalize CoT reasoning.

Definition 2.7. *A **regular relation** is a relation $\phi \subseteq \Sigma^* \times \Delta^*$ that is representable by an FST. If ϕ is a (partial) function, it is called a **regular function**.*

Like all relations, regular relations $\phi \subseteq \Sigma^* \times \Delta^*$ have a trivial inverse $\phi^{-1} \subseteq \Delta^* \times \Sigma^*$.

2.4 Turing Machines

We consider the following definition of a probabilistic Turing machine.

Definition 2.8. *A **probabilistic Turing machine** (PTM) is a two-tape machine with a working tape and an output tape specified by the 6-tuple $\mathcal{M} = (Q, \Sigma, \Gamma, \delta, q_\iota, q_\varphi)$, where Q is a finite set of states, Σ is an output alphabet, Γ is a tape alphabet including the blank symbol $\sqcup$, $q_\iota, q_\varphi \in Q$ are the initial and final states, and $\delta \subseteq Q \times \Gamma \times \Sigma_\varepsilon \times \{L, R\} \times \mathbb{Q}_{\geq 0} \times Q \times \Gamma$ is a rationally weighted transition relation. L and R signify the PTM head moving left (L) or right (R) on the tape after a transition. We write transitions $(q, \gamma, y, d, w, q', \gamma') \in \delta$ as $(q, \gamma) \xrightarrow{y,d/w} (q', \gamma')$. Moreover, we require that for any given $q \in Q, \gamma \in \Gamma$, the weights satisfy $\sum_{(q,\gamma) \xrightarrow{y,d/w} (q',\gamma') \in \delta} w = 1$.*

A transition should be interpreted as follows: When $\mathcal{M}$ in state q, reads γ on the working tape, writes y on the output tape, writes γ' on the working tape, move the head in direction d on the working tape. Each computation step randomly selects a transition according to its weight w.[8]

This definition of Turing machines straightforwardly induces a semimeasure over strings (Nowak et al., 2023, Remark 2.2). It is sometimes easier to prove claims about Turing machines through another equivalent machine (Hopcroft et al., 2001). The probabilistic two-stack pushdown automaton is one example (Nowak et al., 2023). See App. C.3 for an overview.

2.5 Recurrent Neural Language Models

Recurrent neural LMs are LMs whose conditional probabilities are given by an RNN.[9]

Definition 2.9. *An **Elman RNN** $\mathcal{R} = (\Sigma, \sigma, D, \mathbf{U}, \mathbf{V}, \mathbf{b}, \boldsymbol{\eta})$ is an RNN with the following hidden state recurrence:*

$$\mathbf{h}_0 = \boldsymbol{\eta} \qquad (t = 0), \quad (7a)$$

$$\mathbf{h}_t = \boldsymbol{\alpha}\left(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{V}\mathbf{r}(y_t) + \mathbf{b}\right) \quad (t > 0), \quad (7b)$$

where $\mathbf{h}_t \in \mathbb{Q}^D$ is the state vector[10] at time step t, $\boldsymbol{\eta} \in \mathbb{Q}^D$ is an initialization parameter, $y_t \in \Sigma$ is the input symbol at time step t, $\mathbf{r} \colon \Sigma \to \mathbb{Q}^R$ is a symbol representation function, $\mathbf{U} \in \mathbb{Q}^{D \times D}$ and $\mathbf{V} \in \mathbb{Q}^{D \times R}$ are parameter matrices, $\mathbf{b} \in \mathbb{Q}^D$ is a bias vector, and $\boldsymbol{\alpha} \colon \mathbb{Q}^D \to \mathbb{Q}^D$ is an element-wise, non-linear activation function.[11]

Because $\mathbf{h}_t$ hides the symbols consumed by the Elman RNN, we also use the evocative notation $\mathbf{h}(\boldsymbol{y})$ to denote the result of the application of Eq. (7b) over the string $\boldsymbol{y} = y_1 \cdots y_t$. The notation $\mathbf{h}(\boldsymbol{y})$ makes it clear that an RNN LM implicitly defines a language encoder.

Definition 2.10. *A representation-based LM is called an **Elman LM** if its representation function is defined by the hidden state of an Elman RNN $\mathsf{enc}(\boldsymbol{y}_{<t}) \stackrel{\text{def}}{=} \mathbf{h}(\boldsymbol{y}_{<t})$.*

The most common choice for the projection function $\mathbf{f}$ is the softmax, whose limitation is that it implies the LM has full support, i.e., an Elman LM with a softmax projection assigns positive probability to all strings in Σ^*. To construct non-full-support Elman LMs, we instead use the **sparsemax** (Martins and Astudillo, 2016):

$$\mathrm{sparsemax}(\mathbf{x}) \stackrel{\text{def}}{=} \operatorname*{argmin}_{\mathbf{z} \in \boldsymbol{\Delta}^{N-1}} \|\mathbf{z} - \mathbf{x}\|_2^2. \quad (8)$$

In contrast to the softmax function, sparsemax is the **identity** on $\boldsymbol{\Delta}^{N-1}$, i.e., we have $\mathrm{sparsemax}(\mathbf{x}) = \mathbf{x}$ for $\mathbf{x} \in \boldsymbol{\Delta}^{N-1}$.

2.6 Neural Networks and Numerical Precision

All implementations of LMs on modern hardware require representations to be fixed-precision floating-point numbers or arbitrary-precision (rational) numbers. In the case of arbitrary precision, an important consideration in processing strings $\boldsymbol{y} \in \Sigma^*$ is the number of bits required to store the representations and how the number of bits scales with the length of the string, $|\boldsymbol{y}|$. This motivates the following definition of precision.

[8] For more details, see App. C.1.

[9] Throughout this paper, we will focus on Elman RNNs (Elman, 1990) as they are the easiest to analyze and a special case of more common networks, e.g., those based on long short-term memory (LSTM; Hochreiter and Schmidhuber, 1997) and gated recurrent units (GRUs; Cho et al., 2014).

[10] Throughout this paper all vectors are column vectors.

[11] Common examples of $\boldsymbol{\alpha}$ include the Heaviside function $\mathrm{H}(x) \stackrel{\text{def}}{=} \mathbb{1}\{x > 0\}$, the sigmoid function $\sigma(x) \stackrel{\text{def}}{=} \frac{1}{1+\exp(-x)}$, and the $\mathrm{ReLU}(x) \stackrel{\text{def}}{=} \max(0, x)$.

Definition 2.11. *The* ***precision*** $\psi(\boldsymbol{y})$ *of a representation-based neural LM is the number of bits required to represent the entries of* $\mathsf{enc}(\boldsymbol{y})$*:*

$$\psi(\boldsymbol{y}) \stackrel{\text{def}}{=} \max_{d \in [D]} \min_{\substack{p,q \in \mathbb{N}, \\ \frac{p}{q} = \mathsf{enc}(\boldsymbol{y})_d}} \lceil \log_2 p \rceil + \lceil \log_2 q \rceil. \quad (9)$$

We say that a representation-based LM is of

- ***constant precision*** *if* $\psi(\boldsymbol{y}) = \mathcal{O}(1)$*, i.e., if* $\psi(\boldsymbol{y}) \leq C$ *for all* $\boldsymbol{y} \in \Sigma^*$ *and some* $C \in \mathbb{R}$*,*
- ***logarithmically bounded precision*** *if* $\psi(\boldsymbol{y}) = \mathcal{O}(\log |\boldsymbol{y}|)$*, i.e., if there exist* $T_0 \in \mathbb{N}$ *and* $C \in \mathbb{R}$ *such that for all* $\boldsymbol{y} \in \Sigma^*$ *with* $|\boldsymbol{y}| \geq T_0$*,* $\psi(\boldsymbol{y}) \leq C \log_2 |\boldsymbol{y}|$*,*
- ***linearly bounded precision*** *if* $\psi(\boldsymbol{y}) = \mathcal{O}(|\boldsymbol{y}|)$*, i.e., there exist* $T_0 \in \mathbb{N}$ *and* $C \in \mathbb{R}$ *such that for all* $\boldsymbol{y} \in \Sigma^*$ *with* $|\boldsymbol{y}| \geq T_0$*,* $\psi(\boldsymbol{y}) \leq C|\boldsymbol{y}|$*, and*
- ***unbounded precision*** *if* $\psi(\boldsymbol{y})$ *cannot be bounded by a function of* $|\boldsymbol{y}|$*.*

The constructions in the existing literature range from constant to unbounded precision. The RNNs considered by Svete and Cotterell's (2023) encoding of deterministic PFSAs, for example, results in an RNN that is of constant precision. Weiss et al. (2018), Merrill (2019) and Merrill et al. (2020) consider models with logarithmically bounded precision. In contrast, articles treating the Turing completeness of neural networks (Siegelmann and Sontag, 1992; Nowak et al., 2023) require unbounded precision to be able to represent the fact that a Turing machine may fail to halt. Naturally, our constructions of Turing complete LMs will also require unbounded precision.

2.7 Transformer Language Models

Transformer LMs compute the conditional distributions $p(\overline{y}_t \mid \boldsymbol{y}_{<t})$ by means of self-attention. Because transformer LMs necessitate the precise introduction of multiple sub-components, we postpone the full introduction to App. C.4 and only review the basics here. We borrow much of the notation and formalization from Svete and Cotterell (2024).

A transformer is a composition of multiple transformer **layers**, each of which implements the **attention mechanism**. The attention mechanism works as follows. It takes a **query** vector $\mathbf{q} \in \mathbb{R}^D$ and two matrices: The matrix $\mathbf{K} \in \mathbb{R}^{N \times D}$ of **keys** and the matrix $\mathbf{V} \in \mathbb{R}^{N \times D}$ of **values** and computes a weighted average of the value vectors based on the compatibilities of the key vectors to the query vector, as scored by a scoring function f.

Attention weights are computed by normalizing the scores $f(\mathbf{q}, \mathbf{k}_1), \ldots, f(\mathbf{q}, \mathbf{k}_t)$. The choice of the normalization function has concrete implications on representational capacity (Hao et al., 2022; Svete and Cotterell, 2024). We focus on the **hard attention** projection function.

Definition 2.12. ***Hard attention*** *is computed with the* $\mathrm{hardmax}$ *projection function*

$$\mathrm{hardmax}(\mathbf{x})_d \stackrel{\text{def}}{=} \begin{cases} \frac{1}{m} & \textbf{\textit{if}}\ d \in \mathrm{argmax}(\mathbf{x}) \\ 0 & \textbf{\textit{otherwise}}, \end{cases} \quad (10)$$

for $d \in [D]$*, where* $\mathbf{x} \in \mathbb{R}^D$ *and* $m \stackrel{\text{def}}{=} |\mathrm{argmax}(\mathbf{x})|$ *is the cardinality of the argmax set.*

A transformer layer uses the attention mechanism followed by a position-wise MLP[12] to compute augmented representations $\mathbf{z}_t$ of the input representations $\mathbf{X}_t = \left(\mathbf{x}_1^\top; \cdots; \mathbf{x}_t^\top\right) \in \mathbb{R}^{t \times D}$. The query $\mathbf{q}_t$, the keys $\mathbf{K}_t$, and values $\mathbf{V}_t$ are all transformations of the input representations $\mathbf{X}_t$. Initially, $\mathbf{X}_t$ are computed by some static representation function of the symbols and their positions. Multiple transformer layers are stacked into a transformer, which computes the (deep) contextual representations of all symbols in the string. The contextual representation of the final symbol in the string is then used to define the representation function enc of a transformer LM.

3 CoT Reasoning and Weak Equivalence

In this section, we argue that CoT reasoning can be seen as a way of comparing the input language of a formal automaton with the output language of an autoregressive LM. We then introduce a formalization of CoT reasoning that allows us to characterize the representational capabilities of CoT-augmented LMs in terms of their weak equivalence to well-studied weighted automata.

3.1 Equivalence and Augmented Alphabets

Autoregressive LMs generate strings by outputting individual symbols $\overline{y} \in \overline{\Sigma}$ one at a time until EOS is generated. The resulting string $\boldsymbol{y}$ is the output of the LM and the outputs generated in this manner implicitly define the distribution of the LM. Models studied in existing work on the representational capacity of neural LMs, however,

[12] For more details, see App. C.4

often do not (only) emit symbols from Σ. Rather, they also emit symbols from some *larger* alphabet Δ that includes symbols that encode additional information required to simulate a given formal model of computational. For example, the symbols generated by Pérez et al.'s (2021) transformer contain both the output symbols as well as (the changes to) the configuration of the Turing machine.

It is not difficult to see how generating strings from an augmented alphabet can be seen as a form of CoT reasoning. The outputs *not* intended to be a part of the final output can simply be seen as the result of the additional computational steps performed by the CoT-augmented LM. These outputs are later removed in post-processing and only the string with symbols from our target alphabet Σ remains. In this sense, Pérez et al.'s (2021) construction works with a form of CoT reasoning without explicitly mentioning it. This connection was made explicit in concurrent work by Merrill and Sabharwal (2024).

Outputting additional information is not regarded as an issue when the task is to simulate (unweighted) Turing machine runs on a given *input string* because what matters, in that case, is only whether the model accepts or rejects the input (Siegelmann and Sontag, 1992; Pérez et al., 2021, *inter alia*). However, when considering the LM induced by a PTM, we do care about the alphabet the distribution over strings is over. Thus, given an LM over Σ^*, it follows that neural LMs outputting additional information cannot define the same distribution, i.e., they cannot be weakly equivalent. Nevertheless, the ability to output additional information while generating a string seems natural and useful; it is the heart of CoT reasoning. And, indeed, models that can only output symbols that are part of the final string are restricted to doing real-time computation (Weiss et al., 2018; Nowak et al., 2023; Svete et al., 2024).

3.2 Regular Reducibility

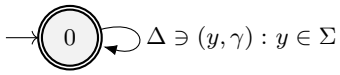
We now formalize the notion of CoT reasoning through the following definition which allows a model to output additional information *not* considered part of the final output:

Definition 3.1. *An LM p over Δ is* ***regularly reducible*** *to a LM q over Σ if there exists a regular function $\phi\colon \Delta^* \to \Sigma^*$ such that $q \circ \phi$ is weakly equivalent to p.*

We can use the function ϕ to map the strings sampled from an LM, ones additionally encoding the intermediary steps of computation, into the final output, i.e., strings $\boldsymbol{y} \in \Sigma^*$.

Definition 3.2. *We say an alphabet Δ satisfies the* ***Σ-augmentation condition*** *for an alphabet Σ if $\Delta \subseteq \overline{\Sigma}_\varepsilon \times \Gamma_\varepsilon \setminus \{(\varepsilon, \varepsilon)\}$ for some alphabet Γ.*

This means if we care about strings from Σ^* and we have an LM over the closure of a Σ-augmented alphabet Δ, then each symbol the LM outputs encodes is either an output symbol from Σ, an intermediate computation symbol from Γ, or both.[13] The computation symbols can then easily be removed by applying the per-symbol projection function $\phi\left((y, \gamma)\right) \stackrel{\text{def}}{=} y$, lifted to a non-length-increasing homomorphism over strings $\phi\colon \Delta^* \to \Sigma^*$ to the LM's output element-wise.[14] This function can be implemented by a simple single-state transducer (making it a regular function):



3.3 Properties of Regular Reducibility

To motivate our formalization, in this section, we show several properties of regular reducibility that will allow us to reason about the representational capacity of CoT-augmented LMs later in §3.4. Particularly, as we will see, there exists a strong connection between regular reducibility and the addition of *non-deterministic* choices in the model. To exemplify this concretely, the next theorem shows that regular reducibility allows us to model non-deterministic PFSAs with deterministic ones, which, as discussed in §2.2, cannot be done with just deterministic PFSAs in general.

Theorem 3.1. *Let $\mathcal{A} = (\Sigma, Q, \delta, \lambda, \rho)$ be a PFSA. Then, there exists a* deterministic *PFSA $\mathcal{A}'$ over the alphabet $\Sigma \times Q$ and with the state space $\Sigma \times Q$ that is regularly reducible to $\mathcal{A}$.*

Proof. See App. E. ■

A similar connection exists for PPDAs (see App. E). Thm. 3.1 captures a crucial property of regular reducibility: It allows us to *simulate* non-determinism with a deterministic device. As

[13] While in practice the CoT and the final output come from the same alphabet, we use separate ones for ease of notation and analysis. This is without loss of generality; see App. D.

[14] A non-length-increasing homomorphism is a function $h\colon \Delta^* \to \Sigma^*$ where $h(\varepsilon) = \varepsilon$, $h(x) = y$ for $x \in \Delta$ and $y \in \Sigma_\varepsilon$, and that further satisfies $h(x_1 x_2 \cdots x_T) = h(x_1)h(x_2)\cdots h(x_T)$.